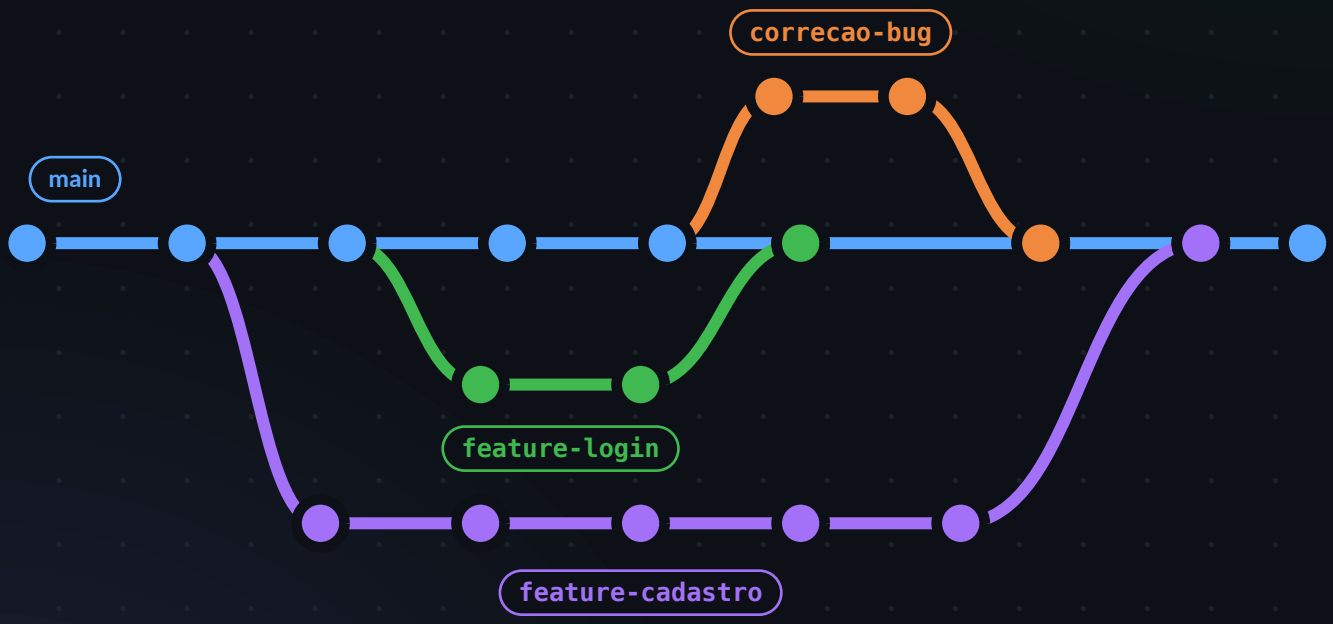
APOSTILA PRÁTICA · PARA INICIANTES

Git & GitHub em equipe

Aprenda o fluxo completo de trabalho com branches, commits, merge, pull e push em um projeto compartilhado.



terminal — meu-projeto

```
$ git pull origin
$ git switch -c feature-login
$ git add .
$ git commit -m "Adiciona tela de login"
$ git push origin feature-login
```

branches · commits · merge · pull · push

Guia de trabalho em equipe

Sumário

Tudo o que você vai aprender, na ordem em que acontece no dia a dia.

01 Introdução . . . 3
02 Entendendo as branches . . . 5
03 Antes de começar a trabalhar . . . 6
04 Criando uma branch . . . 7
05 Fazendo alterações no projeto . . . 8
06 git add . . . 9
07 git commit . . . 10
08 Atualizando sua branch com a main . . . 11
09 Enviando a branch para o GitHub . . . 13
10 Voltando para a main . . . 14
11 Fazendo o merge na main . . . 15
12 Enviando a main atualizada . . . 16
13 Excluindo a branch . . . 16
14 Fluxo completo de trabalho . . . 18
15 Exemplo prático de uma equipe . . . 20
16 Erros comuns . . . 22
17 Checklist do programador . . . 23
18 Cola rápida . . . 24

Como usar esta apostila

Os blocos coloridos aparecem ao longo do texto. Cada cor tem um significado:

IMPORTANTE
Uma ideia essencial para o fluxo fazer sentido.

CUIDADO
Um erro comum de iniciante e como evitá-lo.

DICA
Um atalho ou hábito que facilita o trabalho.

SITUAÇÃO REAL
Uma situação real de equipe: João, Maria e Pedro.

01 Introdução

Antes dos comandos: entenda o cenário

Imagine que **várias pessoas** estão escrevendo o mesmo trabalho ao mesmo tempo. Se todas editarem o mesmo arquivo, vira bagunça. Git e GitHub existem para evitar essa bagunça no mundo do código: cada pessoa trabalha no seu espaço e, no fim, tudo é juntado com segurança.

O que é Git?

O **Git** é uma ferramenta que guarda o **histórico do seu projeto**. A cada etapa importante você tira uma "foto" do código. Se algo der errado, dá para voltar no tempo. Ele roda **no seu computador**, pelo terminal.

O que é GitHub?

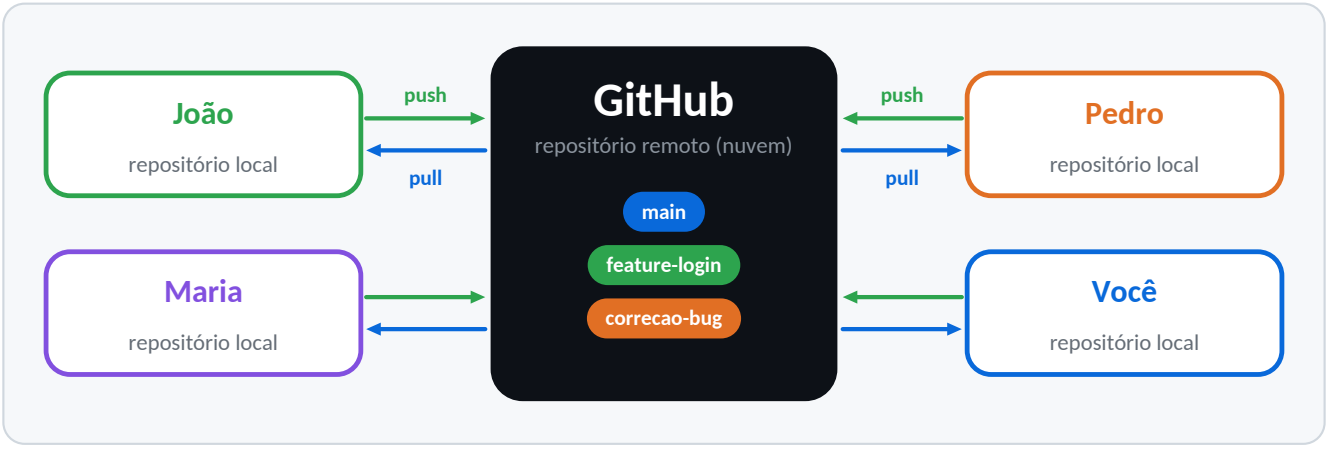
O **GitHub** é um site que guarda uma cópia do projeto **na internet**. É lá que a equipe compartilha o código e enxerga o trabalho uns dos outros.

Git x GitHub: qual é a diferença?

	Git	GitHub
O que é	Um programa (ferramenta)	Um site / plataforma online
Onde fica	No seu computador	Na internet (nuvem)
Para que serve	Controlar as versões do código	Compartilhar e colaborar em equipe
Precisa de internet?	Não	Sim

Resumindo

O **Git** é o caderno onde você registra cada versão do trabalho. O **GitHub** é a estante online onde a equipe toda guarda e consulta esses cadernos.



Cada pessoa tem uma cópia do projeto no próprio computador. O GitHub é o ponto de encontro da equipe.

Por que o Git é importante em equipe?

- Cada pessoa trabalha **sem atrapalhar** as outras.
- Todo o **histórico** fica guardado: quem mudou o quê e quando.
- Erros podem ser **desfeitos** com segurança.
- O código final fica **organizado em um só lugar**.

Vocabulário essencial

Repositório
A pasta do projeto, com todo o histórico guardado pelo Git.

main
A branch principal: a versão "oficial" e funcionando do projeto.

Branch
Uma linha de trabalho separada. Você mexe nela sem alterar a main.

Commit
Uma "foto" salva do projeto, com uma mensagem do que mudou.

Push
Enviar seus commits do computador para o GitHub.

Pull
Baixar do GitHub as novidades que outras pessoas enviaram.

Merge
Juntar o trabalho de uma branch em outra.

origin
O "apelido" do endereço do seu repositório no GitHub.

★ **SITUAÇÃO REAL**

João, Maria e Pedro trabalham no mesmo aplicativo. Cada um cria **uma branch própria**, trabalha em paz e, quando termina, junta o resultado na **main**.

Preparando o terreno (só na primeira vez)

Antes do fluxo, você precisa ter o Git instalado (teste com `git --version`) e ter o projeto no seu computador:

Terminal

```
# diga ao Git quem é você (aparece nos seus commits)
$ git config --global user.name "Seu Nome"
$ git config --global user.email "seu@email.com"

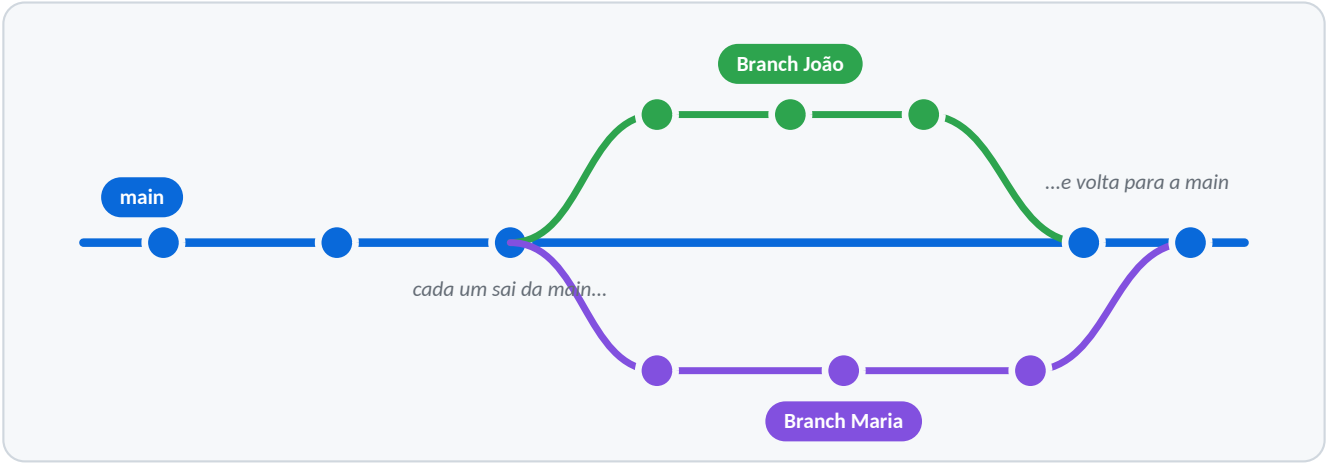
# baixe o projeto do GitHub (só uma vez)
$ git clone https://github.com/equipe/projeto.git
$ cd projeto
```

`git clone` cria uma cópia do projeto no seu computador e já configura o **origin** para você.

02 Entendendo as branches

O conceito mais importante do trabalho em equipe

Uma **branch** (ramo) é uma **linha do tempo paralela** do projeto. Pense numa árvore: o tronco é a **main** e cada galho é uma branch onde alguém trabalha.



João e Maria saem da main, trabalham cada um no seu ramo e depois devolvem o resultado para a main.

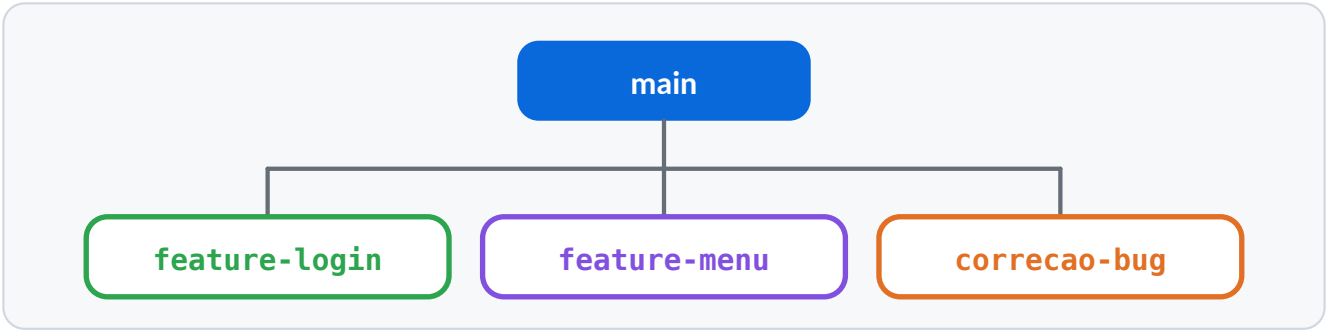
Por que não mexer direto na main?

- A **main** é a versão que funciona. Se alguém quebrá-la, **todo mundo** é afetado.
- Em uma **branch própria** você pode testar, errar e refazer à vontade.
- Só entra na main o que já está **pronto e testado**.
- Várias pessoas trabalham **ao mesmo tempo** sem pisar no trabalho umas das outras.

> **CUIDADO**
> A main deve estar **sempre funcionando**. Evite editar e fazer commits diretamente nela.

Várias branches ao mesmo tempo

Um projeto pode ter muitas branches abertas. Cada uma representa **uma tarefa**:



Três tarefas, três branches, todas saindo da main.

Boas práticas para nomear branches

Bons nomes	Evite
`feature-login`	`teste`
`feature-cadastro`	`minha branch`
`feature-menu`	`joao_novo!!`
`correcao-bug`	`Nova Branch 2`
`melhoria-interface`	`aaa`

- O nome diz **o que será feito** (ex.: `feature-login`).
- Use **letras minúsculas**, sem espaços e sem acentos. Separe palavras com hífen.
- Comece com um prefixo: `feature-` para novidades, `correcao-` para bugs, `melhoria-` para ajustes.
- Nomes curtos e claros: qualquer pessoa da equipe deve entender.

03 Antes de começar a trabalhar

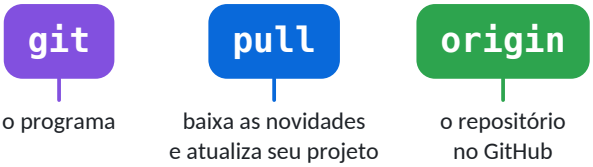
Primeiro passo: atualizar o projeto



Terminal

```
$ git pull origin
```

ANATOMIA DO COMANDO



O que ele faz?

Vai até o GitHub, **baixa as alterações** que outras pessoas enviaram e atualiza o projeto no seu computador.

O que significa origin?

origin é o nome (apelido) do repositório no GitHub. Quando você usa `git clone`, o Git cria esse apelido automaticamente.

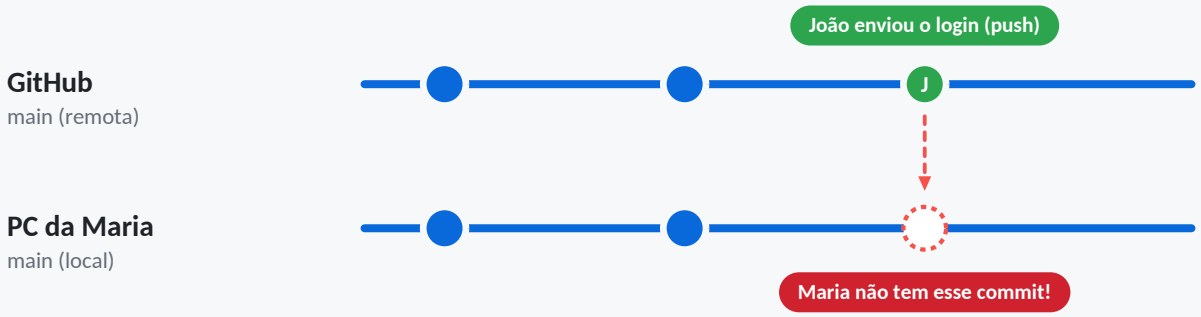
Por que atualizar antes de começar?

Se você começar em cima de um código **antigo**, vai trabalhar sem saber o que os colegas já mudaram. Isso gera retrabalho e **conflitos** depois.

> ✓ **DICA**
> Faça o `git pull origin` estando na **main**. Assim a sua branch nova já nasce atualizada.

Situação real: o que acontece sem o pull?

O **João** terminou o login e enviou para o GitHub. A **Maria** começou a trabalhar **sem atualizar** o projeto: no computador dela, o login do João não existe.



A main do GitHub já tem o commit do João, mas a main da Maria está desatualizada.

Resultado: a Maria vai construir em cima de um código que já mudou. Na hora de juntar tudo, podem surgir conflitos e retrabalho. **O pull evita isso.**

04 Criando uma branch

Seu espaço de trabalho isolado

Terminal

```
$ git switch -c feature-login
Switched to a new branch 'feature-login'
```

ANATOMIA DO COMANDO

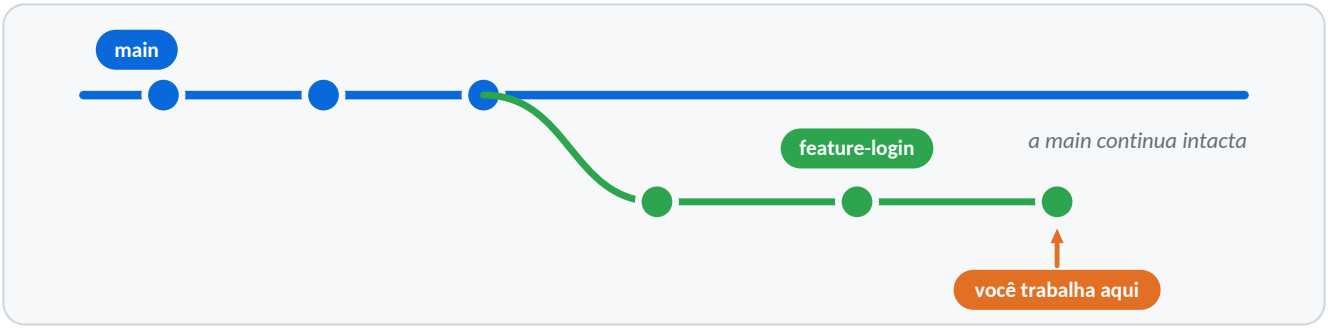
git	switch	-c	NomeBranch
o programa	muda de branch	create: cria a branch	o nome que você escolhe

Esse comando faz **duas coisas de uma vez**: cria a branch nova e já move você para dentro dela. A partir daqui, tudo o que você fizer fica nessa branch.

Cada integrante da equipe cria **a sua própria branch**. Exemplos:

Exemplos

```
$ git switch -c feature-login
$ git switch -c correcao-bug
```



A branch feature-login nasce a partir da main. Você agora trabalha nela.

Como saber em qual branch eu estou?

Terminal

```
$ git branch
  main
* feature-login
```

O **asterisco (*)** marca a branch em que você está. Também vale usar `git status`: a primeira linha mostra a branch atual.

CUIDADO

`-c` cria uma branch **nova**. Para entrar em uma branch que **já existe**, use `git switch NomeBranch` (sem o -c).

05 Fazendo alterações no projeto

Agora é só programar

Com a sua branch criada, você pode **modificar os arquivos normalmente**: criar, editar, apagar. O projeto costuma ter esta cara:



feature-login

Você pode editar, criar e apagar arquivos normalmente.

Tudo isso fica só nesta branch; a main não é alterada.

IMPORTANTE

As mudanças feitas na sua branch **não alteram a main**. A main só muda quando você fizer o merge, no final do fluxo.

Para ver o que você mudou até agora, use:

Terminal

```
$ git status
On branch feature-login
Changes not staged for commit:
        modified:   src/login.js
```

06

git add

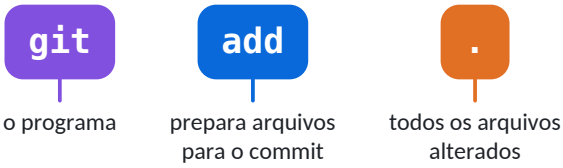
Escolhendo o que vai entrar no commit



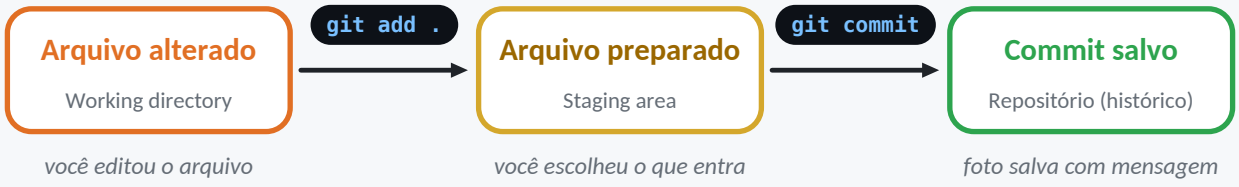
Terminal

```
$ git add .
```

ANATOMIA DO COMANDO



Alterar um arquivo **não** o coloca automaticamente no próximo commit. Antes, você precisa **prepará-lo** com `git add` (a "área de preparação", ou *staging*).



COMO UM PACOTE: 1) você coloca as coisas na caixa (add) 2) fecha e etiqueta a caixa (commit)

O que significa o ponto?

O **.** quer dizer "esta pasta e tudo dentro dela". Ou seja: **todas as alterações** do projeto são preparadas de uma vez. Para preparar só um arquivo:

Terminal

```
$ git add src/login.js
```

CUIDADO

`git add .` leva **tudo**. Rode `git status` antes para conferir se não há arquivos que não deveriam subir (senhas, arquivos temporários).

07 git commit

Salvando uma "foto" do seu trabalho

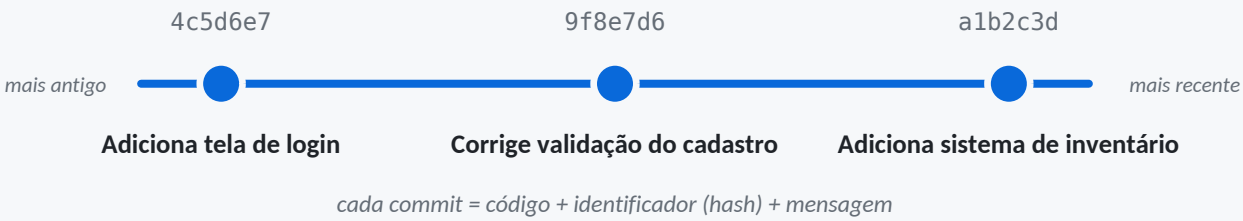
Terminal

```
$ git commit -m "Adiciona tela de login"
```

ANATOMIA DO COMANDO

- `git` — o programa
- `commit` — registra as alterações
- `-m` — message: mensagem
- `"NomeCommit"` — o que você fez, entre aspas

Um **commit** é um ponto salvo na história do projeto: guarda o código preparado e uma **mensagem** que explica a mudança. Se algo quebrar, você sabe exatamente **quando e por quê** mudou.



cada commit = código + identificador (hash) + mensagem

Como escrever boas mensagens

Mensagens ruins	Mensagens melhores
`git commit -m "coisa"`	`git commit -m "Adiciona tela de login"`
`git commit -m "teste"`	`git commit -m "Corrige validação do cadastro"`
`git commit -m "aaaa"`	`git commit -m "Adiciona sistema de inventário"`

- Diga **o que mudou**, não apenas "mexi num negócio".
- Comece com um **verbo**: Adiciona, Corrige, Remove, Atualiza.
- Seja curto: cerca de **50 caracteres** costuma bastar.
- **Um commit = uma ideia.** Evite misturar várias tarefas.

Para ver o histórico de commits:

Terminal

```
$ git log --oneline
a1b2c3d Adiciona sistema de inventário
9f8e7d6 Corrige validação do cadastro
4c5d6e7 Adiciona tela de login
```

CUIDADO

O commit só guarda o que foi **preparado com git add**. Se você esquecer o add, o Git avisa que não há nada para salvar. E lembre: o commit fica só **no seu computador** até o push.

08 Atualizando sua branch com a main

Traga as novidades da main para o seu trabalho



Terminal

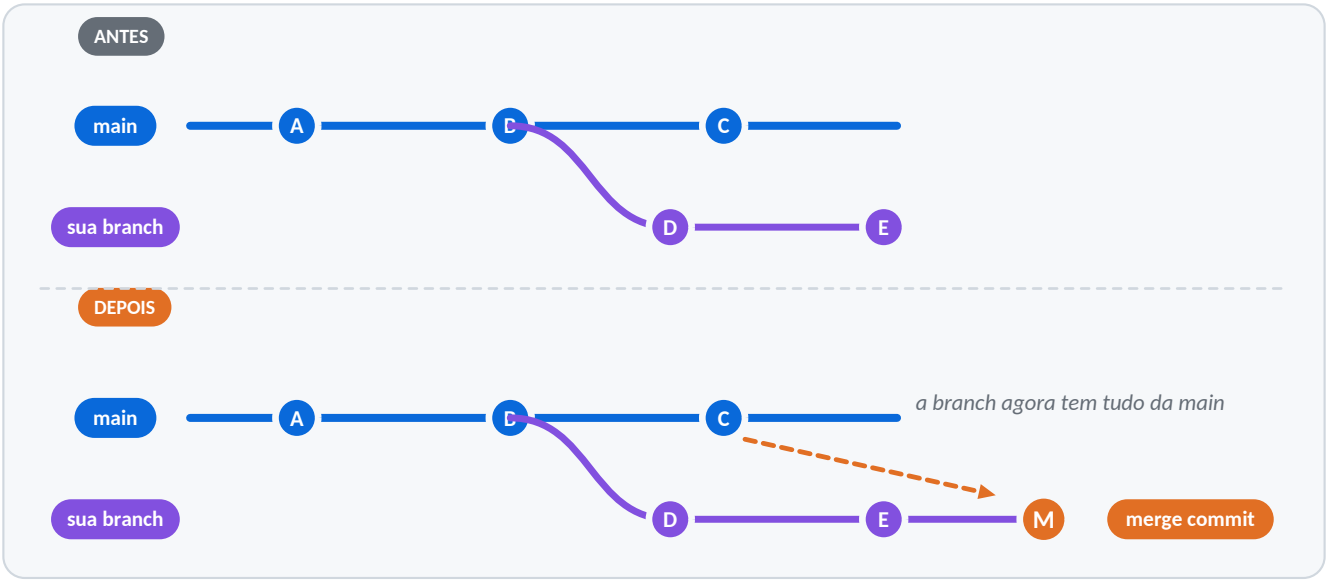
```
$ git merge main
```

ANATOMIA DO COMANDO



Enquanto você trabalhava, outras pessoas podem ter colocado coisas novas na **main**. Com `git merge main` **estando na sua branch**, você traz essas novidades para dentro dela.

Por que fazer isso **antes** de enviar? Se houver conflito, você resolve **na sua branch**, e a main nunca fica quebrada.



Antes: a main andou (C) enquanto a sua branch seguia (D, E). Depois: o merge cria um novo commit (M) com tudo junto.

Terminal

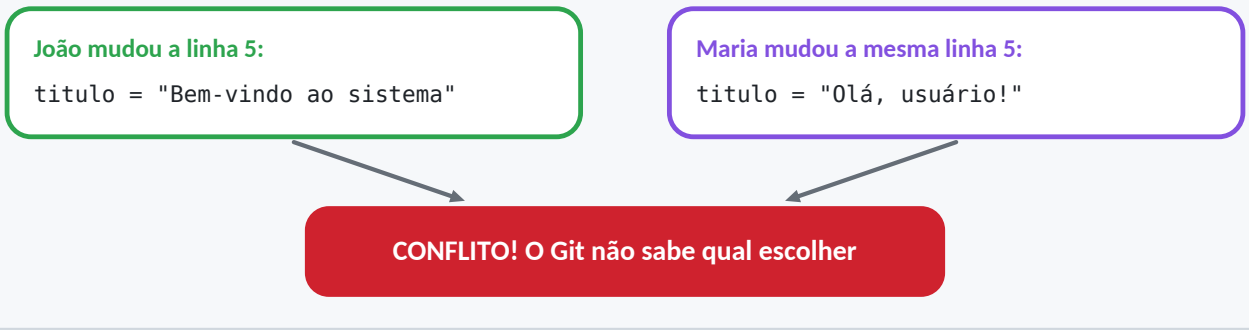
```
$ git merge main
Merge made by the 'ort' strategy.
 src/menu.js | 12 ++++++++++++
 1 file changed, 12 insertions(+)
```

Se a main não tiver nada novo, o Git responde `Already up to date.` e está tudo certo.

> ✔ **DICA**
> Se você demorou dias e a sua main local pode estar velha, atualize antes: `git switch main`, `git pull`, e volte com `git switch sua-branch`.

Conflito de merge

Um **conflito** acontece quando duas pessoas alteram **a mesma parte do mesmo arquivo** de formas diferentes. O Git não sabe qual versão escolher e pede a sua ajuda. **É normal**, e não significa que você errou.



Quando isso ocorre, o Git marca o arquivo assim:

src/login.js

```
<<<<<<< HEAD
titulo = "Bem-vindo ao sistema"
=======
titulo = "Olá, usuário!"
>>>>>>> main
```

Entre `<<<<<<<` e `=======` está a **sua versão**. Entre `=======` e `>>>>>>>` está a versão que veio da **main**.

> **O que fazer quando aparece um conflito?**
> Siga os passos abaixo com calma.

1. **Calma.** Conflito é parte normal do trabalho em equipe.
2. Rode `git status`. Os arquivos em conflito aparecem como *both modified*.
3. Abra o arquivo e procure as marcas `<<<<<<<`.
4. Decida: fique com **uma** versão, com a **outra**, ou **junte as duas**. Depois **apague as marcas** (`<<<<<<<`, `=======` e `>>>>>>>`).
5. Teste o código. Depois rode `git add .` e `git commit -m "Resolve conflito com a main"`.
6. Na dúvida, **converse com quem mexeu naquela parte**. Perguntar é melhor do que adivinhar.

09 Enviando a branch para o GitHub

Compartilhe o seu trabalho com a equipe



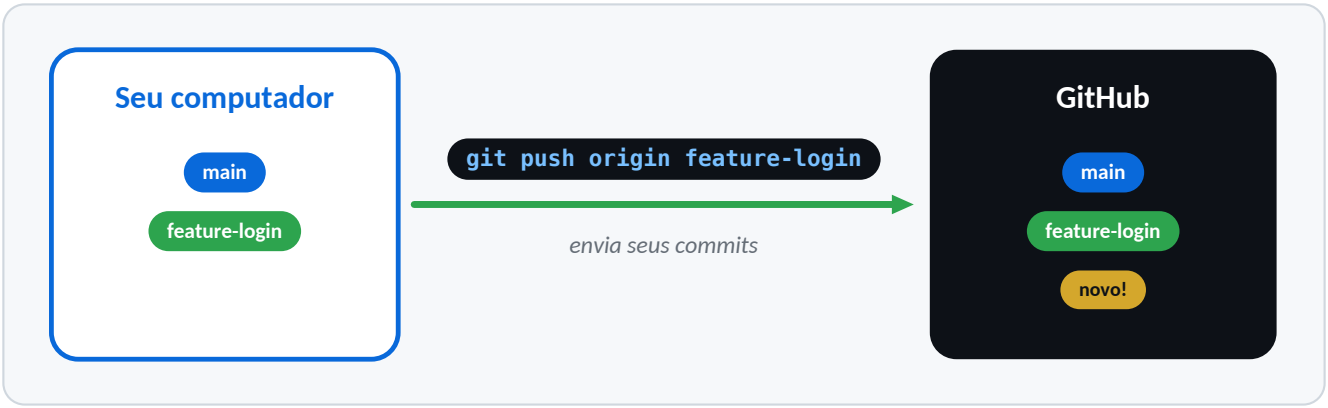
Terminal

```
$ git push origin feature-login
```

ANATOMIA DO COMANDO



O **push** leva os seus commits do computador para o GitHub. Como a branch ainda não existe lá, ela é **criada no GitHub** e passa a aparecer para a equipe toda.



Depois do push, a branch feature-login também existe no GitHub.

Terminal

```
$ git push origin feature-login
To https://github.com/equipe/projeto.git
 * [new branch]      feature-login -> feature-login
```

CUIDADO

O push envia **commits**. Alterações que você só editou (ou só preparou com add) **não vão** para o GitHub.

10 Voltando para a main

Preparando a integração final

Terminal

```
$ git switch main
Switched to branch 'main'
```



O merge sempre acontece **na branch que vai receber** as alterações. Como queremos que a **main** receba o seu trabalho, precisamos estar nela. Repare que agora não usamos o `-c`: a main já existe.

IMPORTANTE

Seu trabalho **não sumiu**. Ele continua guardado na branch `feature-login`. Os arquivos na sua pasta mostram a versão da main até você fazer o merge.

CUIDADO

Confira onde você está com `git branch` (o * marca a branch atual) antes de qualquer merge.

11 Fazendo o merge na main

Integrando a sua branch à versão principal

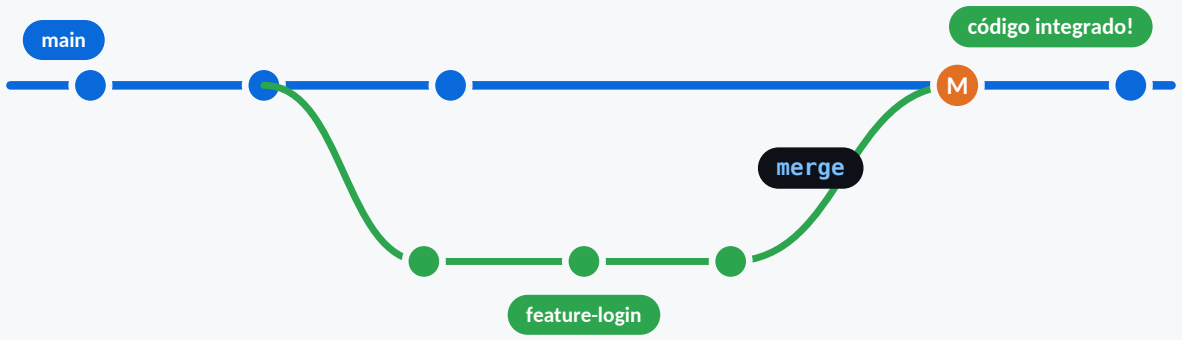
Terminal

```
$ git merge feature-login
```

ANATOMIA DO COMANDO

- `git` — o programa
- `merge` — junta o trabalho
- `NomeBranch` — a branch que será integrada à main

Agora as alterações da sua branch são **incorporadas à main**. Como você já trouxe a main para a sua branch (seção 8), este merge costuma ser **simples e sem conflitos**.



A branch feature-login é integrada à main. O código novo passa a fazer parte da versão principal.

Antes de fazer este merge, confira:

- [ ] O código funciona?
- [ ] Foi testado?
- [ ] A branch está atualizada?
- [ ] Não existem conflitos?
- [ ] O commit está correto?

IMPORTANTE

Em muitas equipes, antes desse merge existe um **Pull Request**: um pedido no GitHub para que alguém revise o código. Nesta apostila fazemos o merge direto no terminal, para você aprender a base do fluxo.

12 Enviando a main atualizada

Entregando o resultado para toda a equipe

Terminal

```
$ git push
```

O merge aconteceu **só no seu computador**. Para o resto da equipe ver a nova versão da main, é preciso enviá-la ao GitHub. Aqui não precisamos escrever o nome da branch: a main já está ligada ao GitHub desde o `git clone`.



O push leva a main atualizada para o GitHub.

CUIDADO

Se o push for **rejeitado** (*rejected*), significa que alguém enviou algo para a main antes de você. Rode `git pull`, resolva o que for preciso e tente de novo.

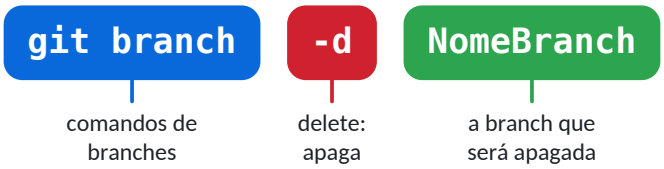
13 Excluindo a branch

Limpando depois do trabalho concluído

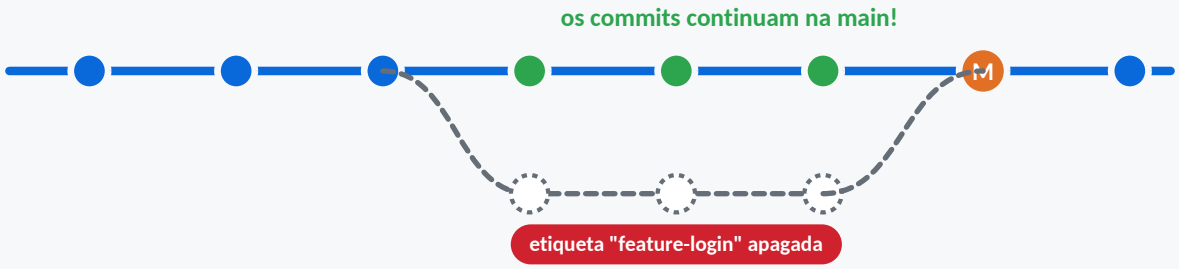
Terminal

```
$ git branch -d feature-login
Deleted branch feature-login (was 3f2a1b9).
```

ANATOMIA DO COMANDO



Depois que a branch foi integrada à main, ela já cumpriu o papel. Apagar mantém o projeto **limpo e organizado**. O `-d` (minúsculo) é a opção **segura**: se a branch ainda tiver trabalho não integrado, o Git recusa e avisa.



Apagar a branch remove só a "etiqueta". Os commits continuam na história da main.

Local x remota

O comando acima apaga a branch **do seu computador**. A cópia no GitHub continua lá. Para apagá-la também (opcional):

Terminal

```
$ git push origin --delete feature-login
```

Você também pode usar o botão **Delete branch** no próprio GitHub.

Exclua quando...	Mantenha quando...
O trabalho já foi integrado à main e enviado	A tarefa ainda está em andamento
A tarefa acabou e ninguém mais vai usar a branch	Alguém ainda vai revisar ou continuar o trabalho

14 Fluxo completo de trabalho

Juntando todas as peças



Fluxo completo

```
# Na main: atualize o projeto
$ git pull origin
# Crie a sua branch e trabalhe nela
$ git switch -c NomeBranch
# ...edite os arquivos do projeto...
$ git add .
$ git commit -m "NomeCommit"
# Traga a main para a sua branch e envie
$ git merge main
$ git push origin NomeBranch
# Volte para a main e integre
$ git switch main
$ git merge NomeBranch
$ git push
# Limpe
$ git branch -d NomeBranch
```

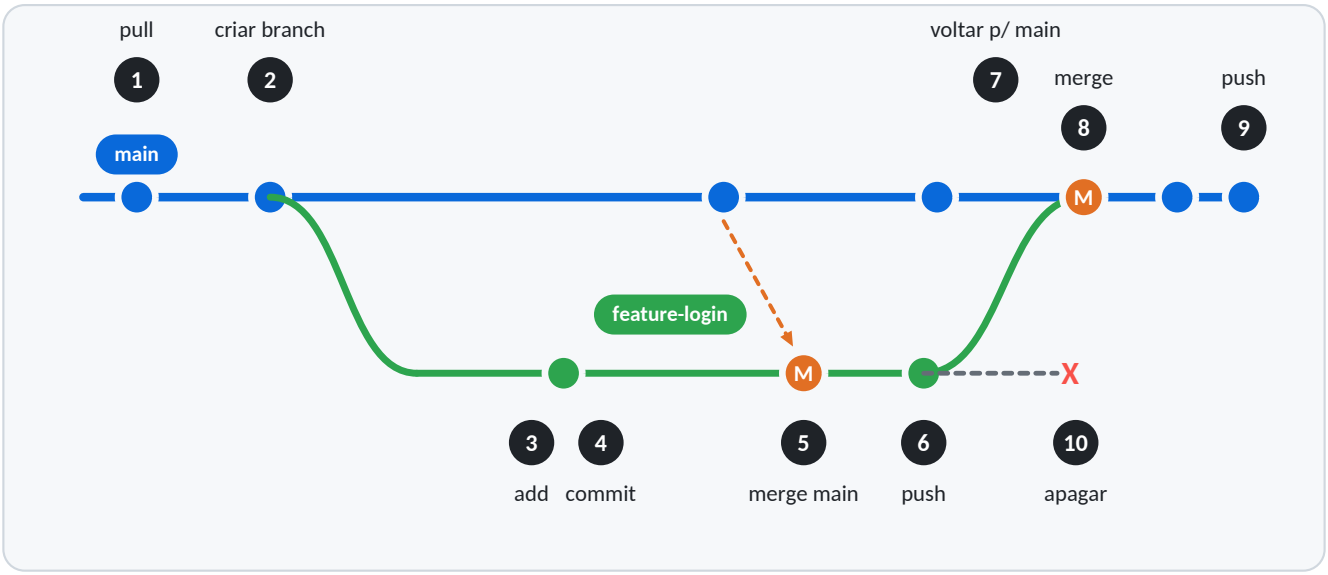
Cada etapa, em uma tabela:

Etapa	Comando	Função	Onde
1	`git pull origin`	Atualiza o projeto	main
2	`git switch -c NomeBranch`	Cria uma branch	main
3	`git add .`	Prepara alterações	sua branch
4	`git commit -m "..."`	Registra alterações	sua branch
5	`git merge main`	Atualiza a branch com a main	sua branch
6	`git push origin NomeBranch`	Envia a branch para o GitHub	sua branch
7	`git switch main`	Volta para a main	sua branch
8	`git merge NomeBranch`	Integra a branch	main
9	`git push`	Envia a main atualizada	main
10	`git branch -d NomeBranch`	Remove a branch local	main

DICA

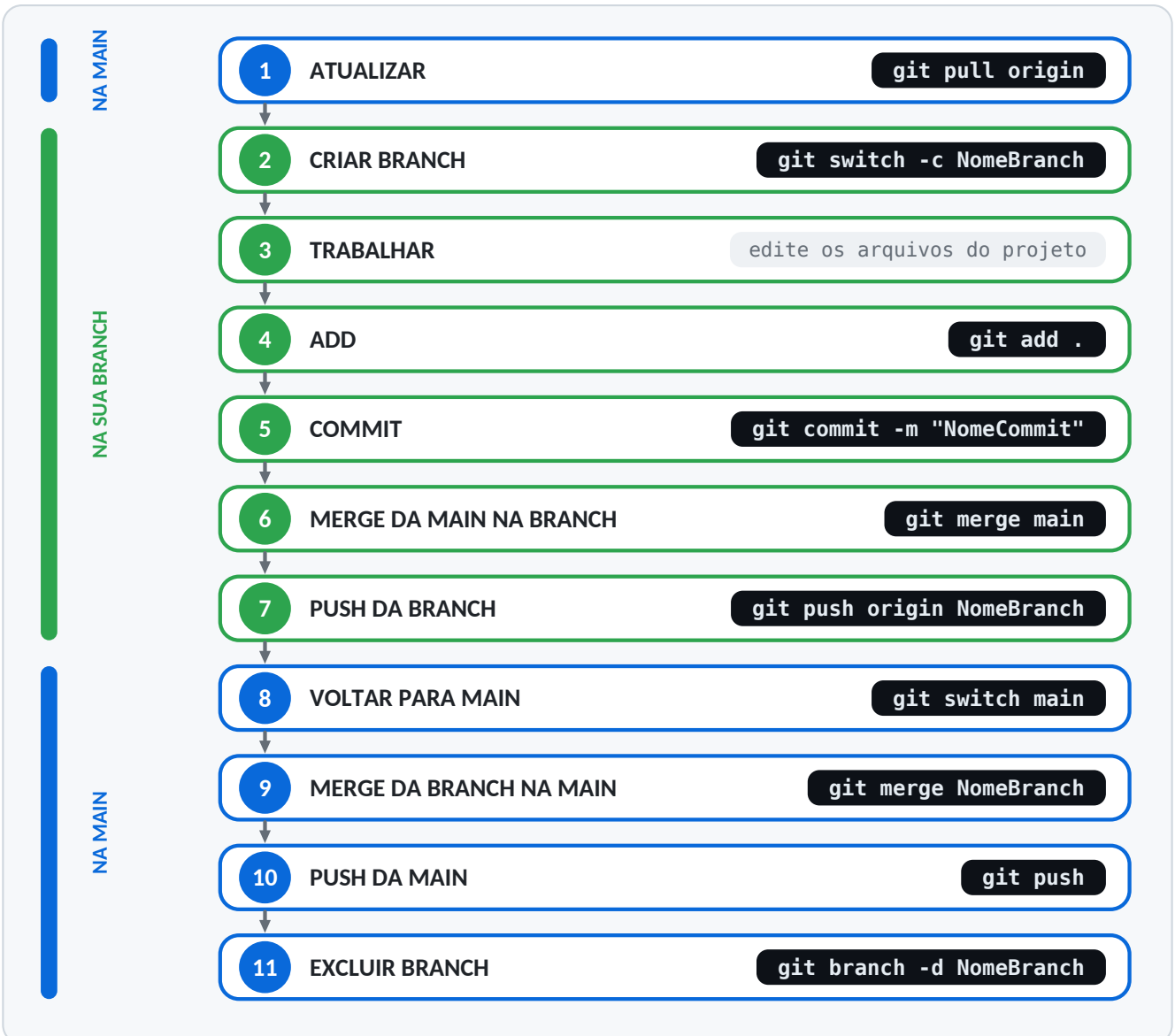
Decorar a ordem importa menos do que entender o porquê: **atualizar** (pull), **isolar** (branch), **registrar** (add + commit), **sincronizar** (merge main), **compartilhar** (push), **integrar** (merge na main) e **limpar** (delete).

O caminho das branches



Os números são as etapas da tabela. O M laranja marca um merge.

O fluxo em uma imagem



15 Exemplo prático de uma equipe

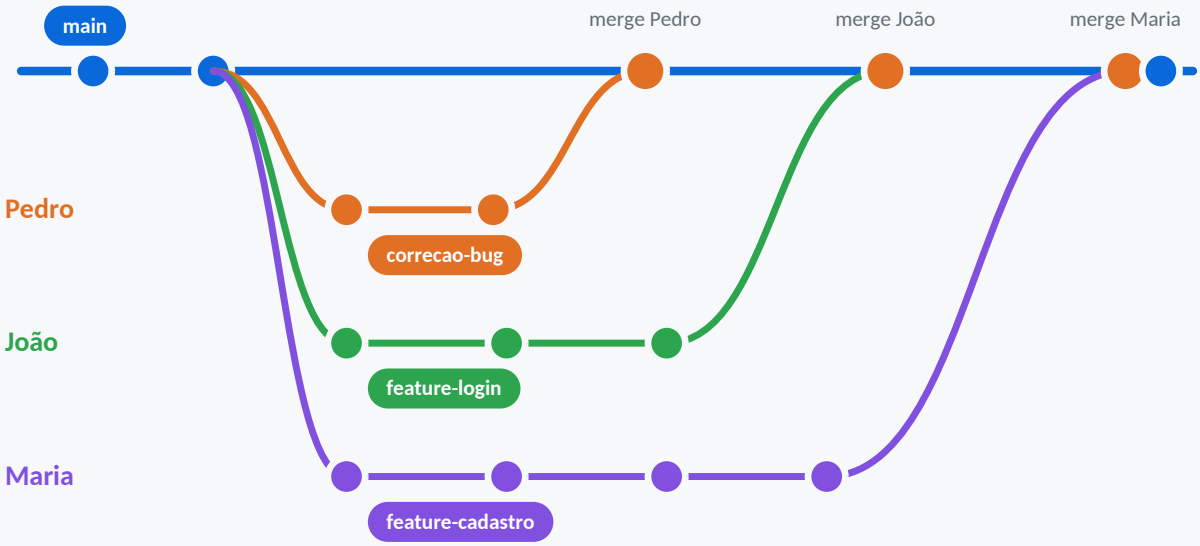
João, Maria e Pedro trabalhando juntos

Uma equipe de 3 pessoas desenvolve uma loja virtual. Cada uma recebeu uma tarefa:

João	Maria	Pedro
sistema de **login**	sistema de **cadastro**	correção de um **bug**

Por que ninguém atrapalha ninguém?

As três branches existem **ao mesmo tempo**. Cada pessoa edita arquivos na sua linha de trabalho, e a **main só muda quando alguém faz o merge**.



Cada pessoa tem sua linha de trabalho; a main só muda quando o merge acontece.

Manhã: cada um cria a sua branch

Todos começam do mesmo jeito: atualizam o projeto e criam **a própria branch**.

Terminal do João

```
$ git pull origin
$ git switch -c feature-login
$ git add .
$ git commit -m "Adiciona tela de login"
```

Terminal da Maria

```
$ git pull origin
$ git switch -c feature-cadastro
$ git add .
$ git commit -m "Adiciona formulário de cadastro"
```

Terminal do Pedro

```
$ git pull origin
$ git switch -c correcao-bug
$ git add .
$ git commit -m "Corrige erro no cálculo do total"
```

Fim do dia: integrando tudo à main

Pedro termina primeiro (bug urgente), depois João e por último Maria. Cada um repete o mesmo caminho de volta para a main. Veja o que a Maria faz:

Terminal da Maria

```
$ git merge main
$ git push origin feature-cadastro
$ git switch main
$ git merge feature-cadastro
$ git push
$ git branch -d feature-cadastro
```

★ **SITUAÇÃO REAL**

Quando a Maria chegou, a **main já tinha** o trabalho do Pedro e do João. Por isso o `git merge main` dela é tão importante: ele **traz esse trabalho para a branch dela**. Se houver conflito, ela resolve ali, e a main continua saudável.

16 Erros comuns

O que todo iniciante já passou (e como evitar)



ERRO 1 · Trabalhar direto na main	
Problema	Você edita arquivos e faz commits na main, sem criar uma branch.
Por que acontece	Parece mais rápido e a gente esquece de criar a branch.
Como evitar	Sempre rode `git switch -c` antes de mexer e confira com `git branch`.

ERRO 2 · Esquecer o git pull	
Problema	Você trabalha em cima de uma versão antiga do projeto.
Por que acontece	A vontade de começar logo faz pular a atualização.
Como evitar	Faça do `git pull origin` o **primeiro comando** de toda sessão de trabalho.

ERRO 3 · Commit sem adicionar os arquivos	
Problema	O commit sai vazio ou o Git avisa que não há nada para salvar.
Por que acontece	Editar um arquivo não o prepara. Falta o `git add`.
Como evitar	Rode `git add .` e confira com `git status` antes do commit.

ERRO 4 · Mensagens pouco descritivas	
Problema	O histórico fica cheio de "teste", "coisa" e "aaaa".
Por que acontece	Pressa na hora de commitar.
Como evitar	Escreva **o que mudou**, começando com um verbo: "Adiciona tela de login".

ERRO 5 · Merge sem atualizar a branch	
Problema	Os conflitos aparecem só na hora de integrar à main.
Por que acontece	A etapa `git merge main` foi pulada.
Como evitar	Sempre traga a main para a sua branch **antes** de enviar e integrar.

ERRO 6 · Esquecer de fazer push	
Problema	Seus colegas não veem nada, o trabalho só existe no seu computador.
Por que acontece	Commit salva localmente. Quem envia é o push.
Como evitar	Termine o dia com `git status` e confirme que tudo foi enviado.

ERRO 7 · Merge na branch errada	
Problema	As alterações vão parar num lugar que você não queria.
Por que acontece	Esqueceu o `git switch main` antes do merge final.
Como evitar	O merge acontece na branch **em que você está**. Olhe o `git branch` antes.

ERRO 8 · Ignorar conflitos	
Problema	Marcas como `<<<<<<<` ficam no código e o programa quebra.
Por que acontece	Medo ou pressa na hora de resolver.
Como evitar	Resolva, **apague as marcas**, teste e só então faça o commit.

ERRO 9 · Apagar a branch antes de integrar	
Problema	Você pode perder trabalho que ainda não estava na main.
Por que acontece	Usar `-D` (maiúsculo) força a exclusão, sem avisos.
Como evitar	Use `-d` (minúsculo) e só apague depois do merge e do push.

17 Checklist do programador

Confira antes de cada etapa



Antes de começar

- [ ] Estou na branch correta?
- [ ] Fiz git pull origin?
- [ ] Criei minha branch?

Antes do push

- [ ] Testei meu código?
- [ ] Fiz git add .?
- [ ] Fiz um commit com mensagem clara?
- [ ] Atualizei minha branch com main?

Antes do merge

- [ ] Estou na main?
- [ ] Minha branch foi testada?
- [ ] Não existem conflitos?

Depois do merge

- [ ] Fiz git push?
- [ ] A main está atualizada?
- [ ] A branch pode ser excluída?

DICA

Imprima ou salve esta página. Percorrer a checklist leva menos de um minuto e evita a maioria dos problemas de equipe.

18 Cola rápida

Os comandos do fluxo, em uma página



#	Comando	O que faz
1	`git pull origin`	Atualiza o projeto com o que está no GitHub.
2	`git switch -c NomeBranch`	Cria a sua branch e entra nela.
3	`git add .`	Prepara todas as alterações para o commit.
4	`git commit -m "Mensagem"`	Registra as alterações com uma mensagem clara.
5	`git merge main`	Traz as novidades da main para a sua branch.
6	`git push origin NomeBranch`	Envia a sua branch para o GitHub.
7	`git switch main`	Volta para a branch main.
8	`git merge NomeBranch`	Integra a sua branch na main.
9	`git push`	Envia a main atualizada para o GitHub.
10	`git branch -d NomeBranch`	Apaga a branch local já integrada.

Comandos de apoio

`git status`	Mostra o que mudou e o que está preparado.
`git branch`	Lista as branches. O * marca a atual.
`git switch NomeBranch`	Entra em uma branch que já existe.
`git log --oneline`	Mostra o histórico de commits resumido.

A ordem do fluxo

Atualizar › Criar branch › Trabalhar › Add › Commit › Merge da main › Push da branch › Voltar para main › Merge da branch › Push da main › Excluir branch